

Metal NAM Gear Players — Neo Edition — Guida Tecnica

Versione 0.2.0 — 120 parametri APVTS

Documentazione tecnica per utenti avanzati, tone-designer e sviluppatori.

Architettura

Host (VST3 / LV2 / Standalone)

```
NAMCustomAudioProcessor (juce/PluginProcessor.*)
    APVTS (parameters: input, output, ir_*, ng_*, gate_*, od_*, dist_*,
           eq_*, depth, reson, hp_*, ln_*, delay_*, chorus_*, flanger_*,
           reverb_*, tremolo_*, output_mode, calibrate_input,
           input_cal_level, quality_scale, os_2x, ...)
    juce::dsp::Oversampling (opzionale 2x, IIR half-band polyphase)
    Left/Right NAMPipeline (juce/NAMPipeline.*)
        SmartGate / Compressore / OD / Dist (PreampFX.h)
        NeuralAudio::NeuralModel (V1/A2) + volume e tone stack
        Amp Sim nativo (GEAR SX / MARCHELLOW, opzionale)
        EQ (src/dsp/EQ.h) + Depth + Resonance + NoiseGate + HP + LN
        presa per l'analizzatore di spettro (coda circolare 4096)
        IRConvolver ×2 (FFT convolution) con bilanciamento
        Delay / Chorus / Flanger / Reverb / Tremolo (PreampFX.h)
    Metronome (juce/Metronome.h) mescolato a valle, frequenza base
    meter: ingresso, uscita, lettore NAM, IR 1, IR 2, compressore
    Async loader (ThreadPool per model/IR con swap atomico non bloccante)
```

Real-time safety

- **Hot path (processBlock) alloc-free:** allocazioni ammesse solo in `prepare()` e nei costruttori.
- **Load asincrono:** `loadModelAsync` / `loadIRAsync` fanno il parsing su un `juce::ThreadPool` e pubblicano il nuovo `NAMPipeline` in `pendingL_/pendingR_` (raw ptr atomico). `processBlock` consuma lo swap non bloccante all'inizio del blocco.
- **Clear e prepare:** `clearModel/clearIR` e `setOversamplingEnabled` chiamano `suspendProcessing(true/false)` per evitare use-after-free con l'audio thread.
- **Modelli senza metadata:** la normalizzazione/calibrazione consulta i flag `HasLoudness/HasInputLevel/HasOutputLevel` esposti dal fork `fabionet/NeuralAudio` (branch `nam-custom-patches`); se assenti, nessuna correzione viene applicata.

Formato preset

`.nampreset` è XML (APVTS state) con l'aggiunta di:

- `<NAMPreset name="..." version="1" lockModel="0|1" category="...">`
- `<ModelPath>`, `<IRPath>` e `<IR2Path>` — percorso assoluto, oppure **nome nudo** di una risorsa incorporata nel binario, risolta da `resolveBundled` (che rifiuta separatori e ..)
- `<Parameters>` con lo stato APVTS completo

I preset devono contenere tutti i parametri. `applyXml` usa `apvts_.replaceState()`, e i parametri assenti dall'albero **non tornano al default**: restano al valore corrente. Un preset parziale quindi non definisce del tutto il suono. Dopo ogni aggiunta di parametri vanno rigenerati i preset di fabbrica — sono passati da 62 a 111, poi 115, ora **120**.

Formati di scambio

Estensione	Contenuto
<code>.nampreset</code>	formato nativo, un <code><NAMPreset></code>
<code>.prs</code>	preset singolo da esportazione

Estensione	Contenuto
<code>.prst1</code>	elenco di preset, con gli asset <code>.nam</code> e <code>.wav</code> incorporati in base64

L'importazione accetta **solo** queste tre estensioni e pretende comunque che la radice sia un `<NAMPreset>` o lo contenga. In un `.prst1` ogni asset è incorporato una volta sola anche se più preset lo usano; al ripristino i file vengono scritti in `~/.config/NAMCustom/Assets` e i riferimenti riscritti. I nomi con separatori o `..` vengono scartati.

I preset di fabbrica sono embedded nel binario tramite `juce_add_binary_data` in `juce/CMakeLists.txt`. Aggiungere un `.nampreset` in `juce/Presets/Factory/` richiede **anche** l'aggiunta alla lista `SOURCES` del binary data — altrimenti il file non viene incluso.

Gain-staging (porting Steve)

Tre parametri APVTS controllano la modalità di output:

- `output_mode` (Raw / Normalized / Calibrated, default Normalized)
- `calibrate_input` (bool, applica correzione input basata su metadata)
- `input_cal_level` (float, dBu, default 12.0)

Logica:

Modello ha	Modalità richiesta	Comportamento
loudness + input/output_level	Calibrated	Correzione pre-model + post-model in dBu
solo loudness	Normalized	Compensazione loudness ~14 dBu di headroom
loudness + input_level (metà)	Normalized	Fallback 50/50: metà correzione pre-model, metà post-model
nessun metadata	qualunque	No-op — il modello esce al suo livello nativo

Motivo del fallback 50/50: senza `output_level_dbu` non sappiamo dove sta il DAC virtuale del modello; splittare la compensazione evita saturazione dell'IR e self-noise amplificato a valle.

Doppio IR

Ogni `NAMPipeline` tiene **due** `IRConvolver`. Entrambi ricevono lo stesso ingresso e scrivono in buffer separati: se se ne usasse uno solo, il primo sovrascriverebbe il segnale prima che il secondo lo legga.

```
wet = (1-bal)·gain1·IR1(x) + bal·gain2·IR2(x)
out = x·(1-mix) + wet·mix
```

Dettagli che contano:

- con **un solo** IR presente il bilanciamento non attenua, altrimenti ruotando il pomello si perderebbe volume senza motivo
- quando il secondo è abilitato viene elaborato **anche a peso zero**, così la sua coda resta viva e ruotare il bilanciamento non produce scalini
- i picchi per IR sono azzerati a monte dello stadio, così in bypass i meter scendono invece di restare fermi
- `ir2_enable` viene acceso dal processore tramite un listener su `channel_mode` quando si passa a Dual-Mono o Stereo; tornando in Mono **non** viene spento

Esclusione automatica della cassa

`NAMPipeline::loadModel` legge `gear_type` dai metadati del modello via `GetMetadata`. Il valore arriva come JSON grezzo, quindi fra virgolette; si cercano le radici `cab` e `rig` per coprire le varianti (`amp_cab`, `full-rig`, `full_rig`, `amp+cab`). I valori osservati sul campo sono `amp`, `pedal`, `amp_cab`, `full-rig`.

Quando `modelHasCab()` è vero, l'editor forza `ir_bypass` a 1 e disabilita i comandi del primo loader. Tornando a un modello senza cassa il bypass viene riportato **com'era prima** dell'esclusione automatica, non forzato a zero.

Analizzatore EQ

`EQAnalyserComponent` disegna spettro e curva sovrapposti.

- **Spettro:** FFT a 2048 punti, finestra di Hann, 24 fotogrammi al secondo. Il segnale viene prelevato da una coda circolare riempita in `NAMPipeline::process subito dopo` lo stadio EQ, quindi si vede l'effetto della curva. Picco per bin con salita immediata e discesa lenta. Finestra utile $-100\dots-20$ dBFS: il livello del singolo bin è molto più basso di quello complessivo, e con 0 in cima lo spettro resta schiacciato sul fondo.
- **Curva:** `FiveBandEQ::responseDB` costruisce biquad temporanei con **gli stessi setter della catena audio** e ne valuta la risposta. La funzione è statica e riceve i valori dei parametri: nessuno stato condiviso col thread audio, e la curva disegnata non può divergere da quella che si sente.
- **Maniglie:** guadagno in verticale per tutte e cinque; frequenza in orizzontale e Q sulla rotellina solo per la banda MID, che è l'unica parametrica. I gesti sono racchiusi fra `beginChangeGesture` e `endChangeGesture`, così l'automazione dell'host registra un tratto unico.

Con guadagno MID a 0 dB la risposta è esattamente 0.00 dB a ogni frequenza per qualunque Q: un filtro a campana senza guadagno è un passa-tutto. Non è un difetto del disegno.

Metronomo

`nam_dsp::Metronome` genera un click da campanaccio: due parziali inarmoniche che decadono insieme più una punta di seconda armonica per l'attacco. Il primo movimento della battuta sale di intonazione e dura di più (835/1235 Hz, decadimento 85 ms) rispetto agli altri (587/845 Hz, 45 ms).

Si mescola **dopo** il downsampling, quindi gira alla frequenza base e non a quella sovracampionata. Una bandiera atomica per movimento alimenta il lampeggio del TAP nell'interfaccia.

Modelli A2 (SlimmableContainer)

Rilevati via `NeuralAudio` (`version > 0.5.4` o `architecture == "SlimmableContainer"` nel JSON del modello). Espongono:

- `HasQualityScaling()` — bool
- `SetQualityScaleFactor(float 0..1)` — RT-safe, pushata dall'editor via APVTS `quality_scale`

Lo slider SLIM sotto il loader NAM e il pomello QUAL sono legati allo stesso parametro; un `juce::Timer` a 8 Hz abilita o disabilita i due controlli in base a `isCurrentModelSlimmable()`.

`quality_scale` sceglie il sottomodello **per soglia**: `ContainerModel::get_index_for_slimmable_size` scorre i submodel e prende il primo con `max_value` superiore al valore. Con due submodel a soglia 0.5 e 1.0, un valore sotto 0.5 seleziona il più leggero. Il **default è 0**, quindi si parte dal più leggero.

Attenzione a due cose:

- non tutti gli A2 sono slimmabili: un WaveNet v0.6.0 non lo è, mentre lo sono sia `SlimmableContainer` sia il WaveNet slimmabile v0.7.0
- `setEnabled(false)` in JUCE rende un widget inerte ma **continua a disegnarlo acceso**: vanno attenuati anche i cursori, non la sola etichetta, altrimenti sembra abilitato lo stesso

IR loader

- Formato: WAV mono o multicanale (viene tenuto solo il canale 0).
- Sample rate sorgente: bounded ($0 < \text{srcRate} \leq 768$ kHz per rifiutare header forgiati).

- Cap frame: `min(totalPCMFrameCount, srcRate × 4)` prima di allocare (evita OOM da header malevolo).
- Cap finale IR: `~1.5 s @ target sample rate`.
- Lettura via `drwav_init_file` streaming (non slurp completo), `drwav_uninit` garantito su ogni path (incluso `bad_alloc`).

FX chain — implementazione

Tutti gli FX in `juce/PreampFX.h` sono classi **header-only, framework-independent** (nessun include JUCE). Ogni classe segue lo schema:

```
struct XxxFX {
    void prepare(double sampleRate, int blockSize); // alloca qui
    float process(float x);                        // hot path, alloc-free
};
```

Nuovi effetti vanno aggiunti nello stesso stile (memoria team `nam-juce-preampfx-framework-independent`).

Build

Requisiti Linux: `cmake 3.15`, GCC/Clang C++17, ALSA, X11.

```
git clone --recurse-submodules https://github.com/fabionet/metal-nam-gear-player.git
cd metal-nam-gear-player
cmake -B build-juce -S . -DCMAKE_BUILD_TYPE=Release
cmake --build build-juce -j 2
rsync -a --delete "build-juce/juce/NAMCustom_artefacts/Release/VST3/NAM Custom.vst3/" "$HOME/.vst3/NAM Cus
```

Note:

- `-DCMAKE_BUILD_TYPE=Release` **obbligatorio**: senza flag esplicito il tipo di build è vuoto → binario non ottimizzato → audio a scatti.
- `-j 2` **massimo su macchine con poca RAM**: `-j` illimitato può causare OOM-kill (exit 137).
- Il target `NAMCustom_LV2` produce Metal NAM Gear Players Neo.lv2/, il target `NAMCustom_Standalone` produce l'app JACK/ALSA.
- **Gli oggetti di NeuralAudio vanno aggiunti ai target finali**. È una OBJECT library i cui oggetti confluiscono nell'archivio SharedCode di JUCE; linkando quell'archivio il linker scarta i membri di cui nessuno referencia un simbolo, e i registratori statici delle architetture NAM non girano mai. Il sintomo è `No config parser registered for architecture: WaveNet` al caricamento di qualunque modello.

Pacchetto Debian

`./packaging/linux/build-deb.sh 0.2.0`

Assembla `metal-nam-gear-players-neo_<ver>_amd64.deb` da un `build-juce` completato: VST3 in `/usr/lib/vst3`, LV2 in `/usr/lib/lv2`, autonomo in `/usr/lib/<pkg>` con collegamento in `/usr/bin`, voce di menu, licenze e guide.

Identità del plugin

	Full	Neo
Prodotto	Metal NAM Gear Player	Metal NAM Gear Players Neo
Codice VST3	...47313830	...4E67704E (NgpN)
URI LV2	<code>urn:fabionet:metalnamagearplayer</code>	<code>urn:fabionet:metalnamagearplayers-neo</code>
Pacchetto	<code>metal-nam-gear-player</code>	<code>metal-nam-gear-players-neo</code>

Il codice VST3 è l'identificatore su cui gli host distinguono i plugin: toccando `PRODUCT_NAME`, `PLUGIN_CODE`, `BUNDLE_ID` o `LV2URI` va verificato che restino diversi da quelli della Full.

Sicurezza (dalla v0.1.x)

- `.nam` corrotti/malevoli: `nlohmann::json throw` catturato → `std::terminate` dell'host evitato.
- WAV IR forgiati: `cap frame + srcRate limitato + reject channels==0/>64` → OOM DoS mitigato.
- Use-after-free UI/audio in `setOversamplingEnabled` e `clearModel/clearIR` → risolti con `suspendProcessing`.
- **UNC path bypass (corretto nei binari dalla v0.2.0)**: `isLocalSafePath` rifiuta i percorsi UNC in entrambe le forme, i percorsi device NT `\\?\` e `\\.\`, e gli schemi di rete `smb`: `nfs`: `afp`: `ftp`: `http`: `https`: `cifs`: `dav`: `davs`: `file`:. Il controllo si applica al modello, all'IR e al **secondo IR**.
- **Importazione preset**: solo le nostre estensioni, radice verificata, e i nomi degli asset con separatori o .. scartati per impedire scritture fuori dalla cartella.

Licenze

- **Opera combinata: AGPL-3.0-or-later**. L'upstream `mikeoliphant/neural-amp-modeler-lv2` è GPL-3.0, ma JUCE 8 è **AGPLv3 / commerciale**: per GPL-3 §13 il derivato combinato viene distribuito AGPL-3.0-or-later (upgrade permesso, downgrade no).
- Motore **NeuralAudio**: MIT (compatibile).
- Framework **JUCE 8**: AGPLv3 / commerciale (Raw Material Software Ltd.).
- **VST3 SDK**: dual GPLv3 / proprietary Steinberg; usato sotto il ramo GPLv3 (compatibile con l'AGPLv3 risultante).
- Asset preset demo bundled: MIT.

Trademark: “JUCE” è marchio di Raw Material Software Ltd., “VST” di Steinberg Media Technologies GmbH, “Neural Amp Modeler” di Steven Atkinson. Il progetto non è affiliato con nessuno dei suddetti.

Contribuire

Issues e PR su <https://github.com/fabionet/metal-nam-gear-player>. Per il submodule NeuralAudio con le patch delle Has-flags, PR contro <https://github.com/fabionet/NeuralAudio> branch `nam-custom-patches`.